



Apple Developer Academy @ BINUS, Bali · Cohort 2027 · Tech track

Computer Science, class of 2028 · Bengaluru, India [Portfolio](#) · [GitHub](#) · [LinkedIn](#) ·
kunalsain0324@gmail.com · +91 85448 88032

Who I am

I'm a twenty-year-old building AI systems that refuse to invent, and I want to learn the half of the craft I don't have.

Right now I'm a **Frontend Engineering Intern at Emergent (YC S24)**. I'm through to **round 2 at MATS**, working on AI safety, and I was selected as an **Activate AI Fellow** — India's first cohort, one of 15 from 500+ applicants. Before the AI work there were two years of community: sixteen months as **Growth & Ops Lead at Hack Club**, running regional technical infrastructure for 500+ participants, and a semester teaching algorithms to 50+ students. I was a selected delegate to **HPAIR 2026 at Harvard**.

Thirty-two repositories. **829 contributions in the last twelve months**. **29 patches merged into 35 open-source projects** — OpenStreetMap's editor, Django's own website, a Bitcoin Lightning wallet, MetaBrainz, Plone, and the Fortran standard library — carrying **12 approvals from 10 different maintainers across 7 organisations**.

What I actually care about

Every system I use will guess, and then deliver the guess in the voice it uses for facts.

That isn't a bug, it's the default — "I don't know" is harder to build than a confident answer, and nobody is rewarded for building it. So that's what I build instead, and I've now done it enough times to admit it's a thesis rather than a habit.

ATTEST refuses to post ₹99.5 lakh it cannot prove, and reports the refusal as the product. **Support Triage** gives the model one sentence inside an unforgeable box and lets a policy validator decide, so its worst case is "escalate to a human" and never a destructive action. **Recall** keeps no model in its production path, so asking your own memory the same question twice can't return two different pasts. **Kunal HQ** awards me points only from verified data, so I can't lie to it.

I didn't plan that symmetry. I noticed it months later, looking at everything side by side, and concluded I only trust a system that is structurally incapable of flattering me.

The five

Not five apps. Five parts of the same person.

01	AI Safety — the direction I'm chasing	MATS round 2 · Activate AI Fellow · ATTEST, Support Triage, Watershed as the proof
02	Recall — the thing I've built most carefully	A local-first continuity OS. Seven layers, 1,003 files, shipped as a real .dmg
03	Upstream — work strangers graded	29 merged patches, 10 maintainers, four languages including Fortran
04	Community — the half a repository can't show	Hack Club 500+ · 50+ students taught · two ongoing club roles · HPAIR
05	Kunal HQ — how I hold it together	A personal life OS with a game layer that cannot be cheated

Why I want this, specifically

Apple has always been the dream — not the company as an employer, the standard. The insistence that engineering and design are not two departments, and that a product isn't finished when it works. I already build to a version of that alone, with nobody checking. I've never been taught it.

I'm barely inside the door already: I was selected as an **Apple Developer Initiative participant**, which is where I first saw product engineering treated as a discipline rather than a side-effect of shipping.

And here is the honest gap. Everything I've built lives on a screen someone is already sitting in front of — a terminal, a browser tab, an operations dashboard. **I have never built for the device someone holds while something is happening to them.** Three times now, the hardest part of an engineering project has turned out to be a design problem I backed into without the vocabulary for it: ranking in Recall was an editorial question, not a mathematical one; fidelity in Minutes came from a token system, not from matching screens; and the hardest thing in ATTEST is not the solver, it's making an operator understand in one glance why the machine held ₹30,107.39.

An honest "I don't know" is worth nothing if nobody can read it. That's the sentence I'd put on the whole application. The Academy is the only place I know that teaches the missing half as a discipline, in a cohort, over ten months, with people from fields I don't have — and Bali is the campus an international learner can actually reach.

Everything else, briefly

Live and clickable: [Recall](#) · [PDFChat](#) — page-cited answers or an explicit refusal. **Also built:** [Minutes](#), a meeting workspace rebuilt from the product outward · [Typeahead](#), 120k real queries over a 3-node Redis ring · [Web Agent](#), perceive → decide → act · an HTTP/1.1 server on raw sockets with path-traversal and

Host-header defences · a LangGraph multi-agent story pipeline · a Canvas RPG with ERC-721 characters.

Experience: SWE Intern at Ostris AI (shipped auth and verification end to end) · Operations Lead, Open Source Club · Head of Immersions, Aspirants Relations. **Languages:** Python, TypeScript, JavaScript, Java, SQL, C++, Go — and enough Fortran to land a patch in a numerical standard library maintained by scientists. Learning Rust by writing ATTEST's solver kernel.

My portfolio site is hand-written HTML, CSS and vanilla JS. No framework, no build step, one page, one script — and every screenshot on it is a real deployment captured at 1280×800.



AI Safety — the direction I'm chasing

Self-initiated · Solo · Ongoing · MATS round 2 · Activate AI Fellow · Python · Rust · RL

I'm through to **round 2 at MATS**, and I was selected as an **Activate AI Fellow** — India's first cohort, one of 15 from 500+ applicants. I read alignment work the way other people read the news. Three systems I've built are already about the same problem, and I only noticed that afterwards.

The question I keep circling

Every system I use will guess, then deliver the guess in the voice it uses for facts.

That's not a bug. It's the default, because "I don't know" is harder to build than a confident answer, and nobody is rewarded for building it. A reconciliation engine picks one of four possible answers and posts it. A support agent takes a destructive action nobody authorised. The totals still balance, no alarm fires, and nobody finds out until it's expensive.

So the question underneath all my work turns out to be one question: **when a capable model proposes something, what is allowed to believe it?**

Not "is the model accurate." Accuracy is a property you measure afterwards. The thing that decides whether a system is safe to deploy is *structural* — what the model is architecturally permitted to cause. That's the question MATS is about, and it's the question I'd been building around for a year without the vocabulary for it.

Three systems, one architecture

	ATTEST	SUPPORT TRIAGE	WATERSHED
Domain	Financial reconciliation	Adversarial support tickets	Multi-agent resource negotiation
The model's job	Propose which records to look at	Return one typed JSON analysis	Learn a negotiating policy
What it may NOT do	See an amount. Authorise anything.	Decide escalation. Execute a tool. See raw PII.	— it's the object of study
Who decides	A verifier that shares no code with the search	A policy validator adjudicating a schema	measured, not trusted
Result	₹0.00 wrongly posted over 500 held-out settlements	132/132 evaluation tests; byte-identical reruns	100k+ episodes; alliances formed on their own
Stack	Python · Rust kernel	Python · LlamaIndex · BM25	Python · TRL · GRPO · FastAPI

The same move appears in all three: **separate the thing that proposes from the thing that decides, and make the separation structural rather than polite.**

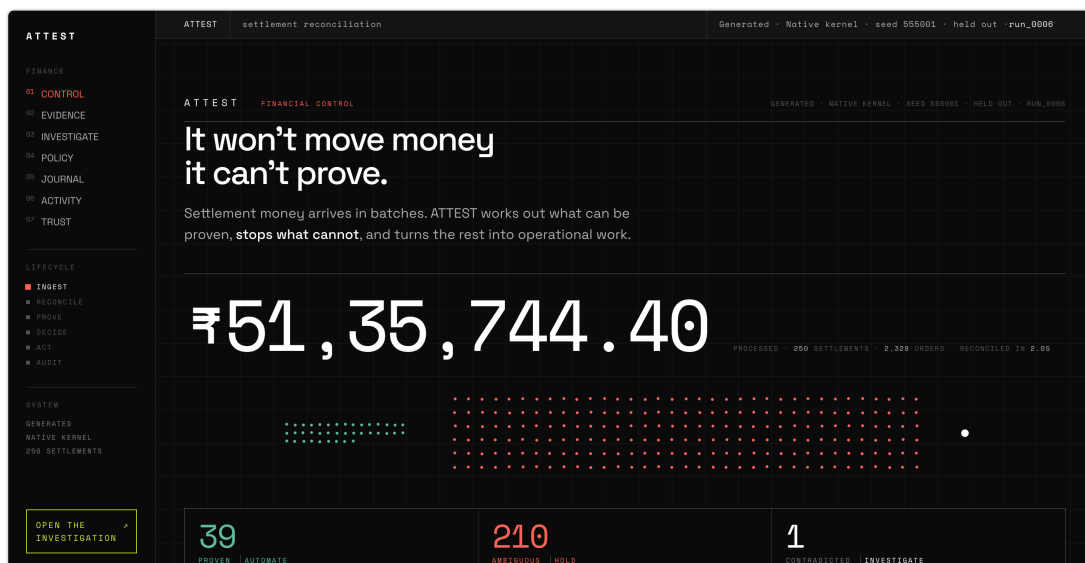
ATTEST — an advisor that cannot move money

github.com/kunalKumar-13/attest

A payment gateway sends one lump sum covering many orders. Where the records carry no order-level reference, the composition has to be derived — and **more than one set of orders can add up to the same credit exactly**. Those sets discharge different customers. Guessing marks the wrong customer as paid *while the books still balance*.

ADVISOR	proposes which records to look at — given names and dates, never an amount
VERIFIER	re-derives the explanation from source records, without reading the advisor
POLICY	automates only where measured error, priced at its 95% bound, beats a human check
LEDGER	records nothing the verifier has not independently proven

Across the held-out panel the advisor is right **27 times in 63**. A 43% opinion is not evidence — so **82.4% of settlements are correctly refused**, ₹99.5 lakh is deliberately held, and of the 33 that auto-posted, 33 were exact. The benchmark table names a matcher *more precise than mine*, because a comparison curated in your own favour proves nothing.



ATTEST’s control view on held-out seed 555001 — 250 settlements: 39 proven and eligible to automate, 210 ambiguous and held, 1 contradicted.

The operator’s view. The held items are the main column, not a footnote — each carries the one field that would resolve it.

Support Triage — the model gets one sentence, inside a box

github.com/kunalKumar-13/deterministic-support-triage

Code owns the control flow. **The LLM is invoked exactly once per ticket, inside an unforgeable sentinel block**, and returns a typed JSON object a policy validator adjudicates. It cannot decide escalation, cannot execute tools, and never sees raw PII or internal control text. Untrusted ticket text never leaves its <<<TICKET>>> frame; retrieved corpus chunks are grounding, never instructions.

Defends against direct and **multilingual prompt injection** (EN/ES/FR/DE/HI/ZH/AR) and obfuscation via zero-width characters, leetspeak, Cyrillic homoglyphs and base64. When retrieved chunks contradict each other, confidence *drops* and the validator escalates. Confidence is Brier-calibrated, escalations capped at 0.60, and it never reports ≥ 0.95 .

Worst-case behaviour is “**escalate to a human**” — **never a destructive action**. Two consecutive runs produce a byte-identical CSV, verified by SHA-256.

Watershed — what happens when nobody is constraining them

A multi-agent reinforcement-learning arena for allocating a scarce resource under real environmental data. Trust dynamics and coalition formation are modelled into the reward; a language model is trained with **GRPO across 100k+ self-play episodes** to study emergent behaviour under competing incentives.

Run it long enough and the agents stopped simply competing. **Alliances formed**. Nobody put them there.

That’s the other half of the same question. ATTEST and Support Triage are about what you permit a model to cause. Watershed is about what appears when you don’t.

Why this belongs in an Apple Developer Academy application

I’m not asking the Academy to teach me machine learning. I’m telling you what I care about, and why the thing I want from you is the missing half of it.

Everything above is invisible. It’s architecture — separations, boundaries, refusals — and it lives in a terminal or a CSV. **None of it ever has to be understood by the person it protects.** The hardest problem in ATTEST isn’t the solver; it’s that the operator has to grasp, in one glance, why the machine held ₹30,107.39 and what single field would release it. I built a control view for that and I know it isn’t good enough.

That’s the gap. I can build a system that knows what it doesn’t know. I can’t yet build the interface that makes a person *feel* that, on a device in their hand, in the four seconds they’re willing to give it. The Academy is where that’s taught as a discipline, with the engineering held to the same standard.

An honest “I don’t know” is worth nothing if nobody can read it.

What I learned

Safety is a shape, not an intention. I couldn’t decide to be careful. Refusing safely required the verifier to be *structurally incapable* of hearing the advisor, the money layer incapable of rounding, the LLM incapable of reaching a tool. Every one of those is an architecture decision made months before the first refusal fired. Build it into the shape of the system, or it’s a promise in a README.

Publishing the number that beats you is what makes the rest believable. Leaving a more-precise competitor in ATTEST’s benchmark cost me the clean story and bought something better: a reader can check my worst comparison and find I ran it first. I no longer ask “are my results good.” I ask **did I go looking for the result that would embarrass me, and did I print it.**

Recall

Self-initiated · Solo · Python · PyQt6 · FastAPI · ChromaDB · Tauri · Chrome extension

[Live](#) · [Code](#) · [Docs](#)

The Broken System

Someone taps you on the shoulder at 3pm. When you sit back down, everything is still there — the file, the tab, the branch. And none of it helps.

Because you didn't lose the files. You lost **continuity**: the state of mind that held them together. You knew why that PDF was open. Now you don't. So you rebuild it by hand, badly, and the climb back costs more than the interruption did.

The tools we have all assume the wrong thing. Spotlight, Windows Search and `grep` assume **you remember a string**. A cognitive restart never asks “which file contains the word `tokenizer`.” It asks *what was I doing, which conversation was that in, where did I leave the implementation*.

Nothing on the machine answers those questions. Every tool answers the one question you are least able to ask.

This is not a search problem. It is a memory problem. And memory has structure that search throws away.

Three Kinds of Forgetting

You never forget one thing. You forget at three different scales at once, and a system that answers only one of them leaves you stranded.

 THE MOMENT	 THE SESSION	 THE MICRO-CONTEXT
A single thing you touched	A block of time you worked in	The topic you were actually chewing on
<i>“that arxiv page, yesterday”</i>	<i>“that evening I spent on reward shaping”</i>	<i>“the four visits that were all the same thought”</i>
Recoverable by keyword — barely	No keyword exists for it	No keyword exists for it
Spotlight can find this	Spotlight cannot	Spotlight cannot
Gap: it's the least useful of the three	Gap: the unit you actually schedule around	Gap: the unit a human actually remembers

These are three different questions, and one ranked list of filenames answers none of them properly. So Recall returns all three, in one list, from one query.

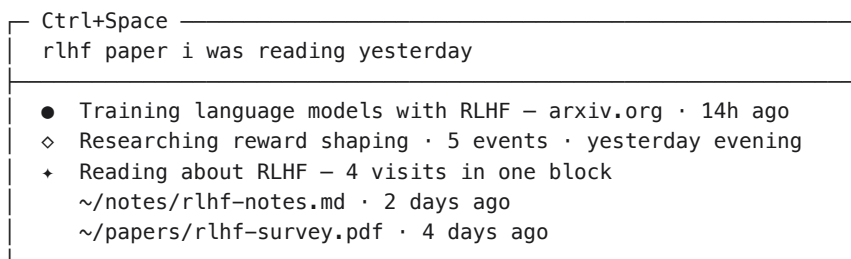
How the System Works

Seven layers in a single process, each composing on the one below it:

Events	per-day JSONL in ~/.recall/events/	"what did I touch?"
Sessions	~30-minute activity groupings	"what was I doing then?"
Micro-contexts	topic-coherent splits of a session	"what was I chewing on?"
Resurfacing	the quiet idle-launcher digest	"what should I notice?"
Threads	persistent topic identity over weeks	"what's the shape of my work?"
Evolution	chronological phases inside a thread	"how did this change?"
Recovery	resumable work, one-click restoration	"what do I open right now?"

A Chrome extension — 200 lines — posts page visits, searches and chat sessions to a local API at 127.0.0.1:4545. watchdog watches the folders you choose; save a file and it's reindexed within ten seconds. Semantic file search runs all-MiniLM-L6-v2 over a ChromaDB index persisted to ~/.recall/index/.

Then press **Ctrl + Space** and type a fragment.



Moment, session and micro-context, in that order, from one half-remembered phrase.

Open it with an **empty** query and it does the thing the product exists for: a quiet *Continue where you left off* card, **capped at four items**, no badges and no feed. Click one and the files, tabs and chats come back **in the order that re-grounds the work** — not the order you happened to open them.

You step back in. You don't rebuild.

The Design Decision — Why there is no LLM in the production path

The instinct is obvious: dump every event into a vector store, put a model on top, let it summarise what you were doing. It demos beautifully. It's the wrong call, on two counts.

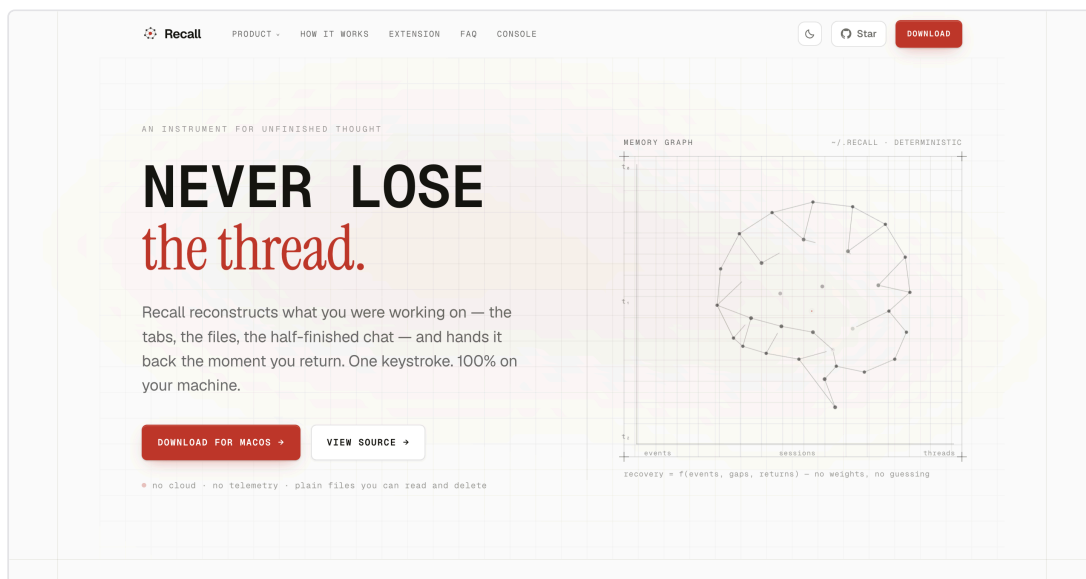
Determinism. The same events in must give the same output out. If a probabilistic ranker sits between you and your own memory, you can ask the same question twice and get two different pasts. A memory tool that is *sometimes* right is worse than none, because you lose the ability to tell which one you're holding. Every layer of the engine is deterministic, and a **35-section smoke test** exists to enforce exactly that contract.

Privacy you can check rather than be promised. This product reads your browsing, your files and your conversations — the most invasive permission set software can ask for. “We take your privacy seriously” is not an answer. So: no cloud, no account, no telemetry, no model call in the production path. Everything Recall knows about you lives in `~/recall/` as plain JSON and JSONL — **readable with `cat`**, **deletable with `rm`**. Delete the folder and it’s a complete reset. You don’t have to trust me; you can go look.

There’s a third reason, and it’s the one I’d defend hardest. **A chatbot answers questions. It does not restore state.** Recall never summarises your work or talks to you. You don’t want a conversation *about* your work — you want to be back inside it.

Those constraints removed the easy version of nearly every feature. That was the point.

The Interfaces



Recall’s shipped site and download.

Recall comes in two shapes, and it’s worth knowing which you’re looking at.

Run from source — one Python process: the PyQt6 launcher in `app/ui/`, plus the FastAPI service in `api/`, in a venv. Full stack, semantic file search included. Three hard layers with no leakage: `app/core/` is pure Python and imports no Qt; the UI talks to retrieval *only* through an HTTP client.

The shipped macOS app — a Tauri menu-bar launcher carrying a headless `recall-daemon` sidecar, which starts only if nothing already answers on `127.0.0.1:4545`. **~26 MB**, because the bundle leaves out PyQt6 and the embedding stack; semantic file search honestly reports *disabled* there, and the browser extension becomes the primary capture path.

Both speak the same `/v1/*` surface, so every claim holds for either.

The Proof

It ships. Not a repo — a `.dmg`, a documentation site at `docs.recall.computer`, a `SUPPORTED_PLATFORMS.md` that states where it doesn't work, a `SECURITY.md`, a changelog and a roadmap. **1,003 files.**

The bar I set before starting was a **three-minute install: five commands to a working launcher**. A continuity tool that takes an afternoon to configure will never be there on the afternoon you need it.

And the honest one: the shipped bundle reports semantic search as *disabled* rather than quietly returning worse results. A degraded mode that announces itself is a feature. A degraded mode that stays silent is a lie with a nice install size.

Why this matters for the Academy

Recall is the project that taught me what I'm missing.

The engine was never the hard part. **Ranking was** — and ranking turned out to be an editorial question, not a mathematical one. The system can surface hundreds of relevant events, and showing you hundreds of relevant events is identical to showing you nothing. Getting to *four items, no badges, no feed* meant repeatedly deciding what **not** to show, and every one of those was a judgement about what a person needs in the ten seconds after they sit back down.

Same with the launcher being one keystroke and one input: at the moment you need it, you are already frustrated, and any interface that asks you to make a choice has failed.

I arrived at both of those by instinct and a lot of throwing things away. **They are design decisions, and I have never been taught how to make them.** That is precisely what I'm asking the Academy for — and Recall is the evidence that I'll use it, because I've already spent a year running into the wall where engineering stops being the answer.

What I Learned

The hard problem was ranking, and ranking turned out to be editorial, not mathematical. The engine can surface hundreds of relevant events, and showing you hundreds of relevant events is identical to showing you nothing. Getting to *four items, no badges, no feed* meant repeatedly deciding what **not** to show — and each of those was a judgement about what a person needs in the ten seconds after sitting down, never a threshold I could tune. I went in thinking retrieval was the project. Restraint was the project.

Constraints adopted on purpose make the product; constraints discovered later make the backlog. Ruling out the cloud and ruling out the model cost me the easy build of almost every feature — and produced the only two claims about Recall a stranger can verify themselves in under a minute. I now write down what software is *not allowed to do* before I write down what it does.



Self-initiated · Solo, but entirely in public · Python · JavaScript · TypeScript · Fortran

[Every upstream pull request →](#)

*Figures verified against the GitHub API on 11 September 2026: **29 merged, 44 open, across 35 repositories.***

The Broken System

Every other project in this portfolio is work I graded myself.

In a personal project, “done” is whatever I decide it is. I choose the problem, the standard, the definition of finished, and the moment to stop. That’s fine — it’s how you learn quickly — but it means the entire evaluation loop runs inside my own head, and a portfolio built only from that is a portfolio of self-assessment.

Upstream, “done” is when someone who has maintained a codebase for a decade — who did not ask for my patch, owes me nothing, and will be maintaining my code long after I’ve moved on — decides it’s good enough to carry.

That is a completely different standard, and it’s the only one here I couldn’t grade in my own favour.

It’s also the closest thing I’ve done to what a ten-month cohort actually is: **sustained collaboration with people who don’t work for you, in a codebase you didn’t write, under conventions you must learn before you’re entitled to an opinion.**

Where the Work Landed

PROJECT	MERGED	LANGUAGE	WHAT IT IS
sugarlabs/musicblocks	8	JavaScript	A music-and-programming microworld for children
django/djangoproject.com	3	Python	The site Django itself runs on
metabrainz/listenbrainz-server	3	Python + TypeScript	Open music listening data
plone/volto	2	JavaScript	The Plone 6 React frontend
ZeusLN/zeus	2	TypeScript	A Bitcoin Lightning wallet
fortran-lang/stdlib	1	Fortran	The Fortran standard library
fortran-lang/webpage	1	Python	Fortran's community site
openstreetmap/iD	1	JavaScript	The OpenStreetMap editor
plone/meta	1	Python	Plone's shared tooling
plone/plone.versioncheck	1	Python	Plone dependency tooling

Plus merged work in three smaller repositories. **Twelve approvals from ten different maintainers across seven organisations.**

The titles say more than the count does:

- musicblocks#8495 — *release the busy state so the widget cannot deadlock*
- musicblocks#8589 — *raise NanError for NaN in every operation that promises it*
- musicblocks#8588 — *give equal19 all nineteen of its ratios*
- plone.versioncheck#79 — *parse buildout files with zc.buildout's parser (instead of by hand)*
- stdlib#1207 — *point the fypp error message at the variable that works*
- iD#12794 — *detect goo.gl shortlinks as an incompatible source*
- zeus#4621 — *BBQr split/join invariants*
- listenbrainz#3856 — *read custom_name, not custome_name*

Almost none of these are features. They are a deadlock, a broken promise in an error contract, a missing ratio, a hand-rolled parser, a licensing trap for mapmakers, and a typo that silently disabled a feature. **This is what maintenance work actually looks like**, and it's the work that decides whether software still functions in five years.

The Design Decision — Going where I'd be uncomfortable

The efficient play is to find one friendly project and file twenty patches. The count climbs fast, the review culture becomes familiar, and you learn one codebase very well.

I deliberately spread across ecosystems that share nothing: a children's education tool, a Bitcoin wallet, a map editor, scientific data QC, Django's own website, and the Fortran standard library. **Four languages, six review cultures, six sets of conventions I had no say in.**

Fortran is the one I'd point at. I had no reason to know Fortran. Numerical scientists maintain `stdlib` to a standard where every patch is read very carefully, and my instincts from JavaScript were worth precisely nothing there. Landing something meant learning how that community argues about correctness *before* writing a line.

That's the skill I was actually practising. Not the language — the ability to arrive somewhere I have no standing and become useful anyway.

The Maintainers

The patches are the visible part. The work was learning what each maintainer needs, and it was different every time.

walterbender — Sugar Labs' org lead. Merged eight. Browser-tested one himself before merging, and pulled two other reviewers onto another. Wants you to act on automated review feedback rather than argue with it.

MonkeyDo — MetaBrainz. Two standing rules, stated plainly: keep the AI-usage disclaimer in the PR template, and run the tests locally before submitting. I discovered the backend suite cannot run on Apple Silicon — the `metabrainz/python` image is amd64-only across all 41 tags, and emulation segfaults. Rather than submit backend work I couldn't test, I moved to the frontend Jest suite, which runs locally. **All three merges came from that side.** The constraint chose the contribution.

sarahboyce — a Django Fellow, so the scarcest time here. Gives itemised reviews and expects every point answered. Pointed me at `django/django`'s own `SeleniumTestCaseBase` as the pattern to follow, which was better than what I'd written.

kaloudis — Zeus's lead. Ran the suite himself on my PR, then gave three concrete suggestions, the sharpest being negative tests for the QR-joining function — the one code path fed directly by untrusted camera input. All three are addressed.

rakshityadav1868 — not a maintainer, another contributor. Reviewed two of my musicblocks PRs carefully and **corrected me on #8594, where he was right and I was wrong.** That one belongs in the list. The correct response to being wrong in public is to say so on the thread and fix it; the correct thing after that is to read his pull requests as carefully as he read mine.

The Proof

Twenty-nine patches now live in software people use daily — a tool children learn music on, the editor behind OpenStreetMap, a Bitcoin wallet, and the Fortran standard library.

But the count isn't the evidence. **Ten maintainers who owed me nothing put their name on my work**, several of them more than once, and several now recognise it when it arrives. That's a relationship, and relationships are how open source actually gets built.

There's a rejection worth naming too: laurent22/joplin closed a patch of mine because the project only accepts PRs from long-term contributors. Not a quality judgement, and the right call for them — but it's the clearest lesson available that a merge is a decision about *trust*, not just about a diff.

Why this matters for the Academy

A ten-month cohort is the same shape as this, and I already know I'm good at it.

Sustained collaboration with people who don't work for you, in a codebase you didn't write, under conventions you have to learn before you're entitled to an opinion — that's open source, and it's also a room of twenty people building together for a year. The skills transfer exactly: reading what someone actually needs before asking them for anything, answering an itemised critique point by point without getting defensive, and conceding in one comment when you're wrong.

It's also the only work in this portfolio I couldn't grade in my own favour. Ten maintainers who owed me nothing put their name on my code. **If you want to know whether I can take feedback from people with a higher standard than mine, that's twenty-nine data points, all public.**

What I Learned

A patch is a proposal to a specific person, and most of the work is understanding who. The same diff gets merged in one project and closed in another, and it is almost never about the code. MonkeyDo needed local test evidence. sarahboyce needed every review point answered individually. walterbender needed me to stop arguing with the bot. Reading what a maintainer actually needs — from their past reviews, before filing anything — improved my merge rate more than any improvement in my code did.

Being wrong in public is cheap; defending it is expensive. When rakshityadav1868 corrected me, the useful move was to concede in one comment and fix it. I'd assumed the cost of being visibly wrong in front of a whole project was high. It's close to zero, and it buys you a reviewer who takes you seriously. What's genuinely expensive is the thirty minutes I once spent defending something I already suspected was wrong.

Community — two years of making things happen for other people

Hack Club · Teaching · Open Source Club · Aspirants Relations · HPAIR at Harvard

Sixteen months running regional technical infrastructure for **500+ participants**. Fifty-plus students taught algorithms. Operations lead of an open-source club and head of immersions for aspirant relations, both current. A selected delegate to **HPAIR 2026 at Harvard**.

Before any of the AI work, this is what I was doing. It's the half of me that a repository can't show.

Why this is in a developer portfolio

A ten-month cohort is not a course. It's twenty-odd people in a room for a year, and whether that room produces anything depends almost entirely on things that aren't engineering: whether the logistics hold, whether people who think differently can hear each other, whether someone notices the person who has gone quiet.

Somebody has to be that person. I've been that person for two years, and I'd rather be judged on it than hide it behind a commit graph.

What I actually did

ROLE	WHEN	THE REAL JOB
Growth & Ops Lead — Hack Club	Nov 2024 – Mar 2026 · 16 months	Regional technical infrastructure for Campfire, Daydream and CodeDay Bengaluru. 500+ participants fed and online.
Teaching Assistant — DSA	May – Jun 2026	50+ students through algorithms and structured thinking. Doubt-resolution and concept clarification.
Operations Lead — Open Source Club	2026 – present	Running the operations of a student open-source community
Head of Immersions — Aspirants Relations	2026 – present	Programming and running immersion experiences for incoming students
Selected Delegate — HPAIR 2026	2026	Harvard Project for Asian and International Relations

Hack Club — 500+ people, fed and online

“Growth & Ops” is a generous title for what the work is. At a 500-person event, nobody remembers the talks. They remember whether the wifi held, whether there was food at 11pm, whether the person who turned up alone found somebody to build with.

Three flagship initiatives across sixteen months — **Campfire**, **Daydream**, and **CodeDay Bengaluru** — and my job was the regional technical infrastructure and the operational logistics underneath all of them. Not the part with a stage.

Here’s what that taught me, and it’s the reason I’m good at the other work in this portfolio: **at that scale, every failure is a failure you didn’t plan for.** The wifi doesn’t go down in the way you tested. So you stop trying to prevent failure and start designing for degradation — what still works when the one thing you were relying on doesn’t, and who finds out first.

That is the same instinct as everything else here. Recall reports semantic search as *disabled* rather than silently returning worse results. Support Triage’s worst case is “escalate to a human.” Kunal HQ’s morning routine has every data source failing independently, so a dead VPN never blocks the weather.

I learned that running events, not writing software.

Teaching — 50+ students, and the third explanation

Fifty-plus students through data structures and algorithms, in doubt-resolution and concept-clarity sessions.

The thing nobody warns you about: the first explanation is the one you already have. The second is the one you prepared. **The third is the one you have to build, live, while someone watches you, out of whatever they just told you they do understand.** That’s the only one that ever works, and you cannot prepare it.

I got better at that over a semester, and it changed how I write. Every README I ship now opens by naming what the software *doesn’t* do — not out of modesty, but because I learned that people can only hold a new idea once they know its edges.

It’s also why I want a cohort rather than a course. I learn fastest with the obligation to explain it to someone who doesn’t already agree with me.

The clubs, and Harvard

Operations Lead of the Open Source Club and **Head of Immersions for Aspirants Relations** — both ongoing, alongside a full-time internship at Emergent and the open-source work. Immersions in particular are the closest thing I’ve run to what the Academy does: put people who don’t know each other into an intense shared experience and make it produce something.

HPAIR 2026 at Harvard put me in rooms with people from completely unrelated fields, arguing about things I had no technical claim on. Every project in this portfolio began as a problem I could name in one sentence, and I'm increasingly sure that's a habit I picked up there rather than from any codebase.

Why this matters for the Academy

Their own guidance says it plainly: *"Ability/potential to be a multi-disciplinary candidate is highly desirable."*

I'm applying as a Tech-track candidate with 29 patches merged into other people's software and three systems built around AI oversight. But **the reason I'd be useful in a room for ten months isn't any of that**. It's sixteen months of being the person who makes the logistics hold, a semester of finding the third explanation, and two ongoing roles that exist purely to make other people's experience work.

The Academy is in Bali, in English, with learners from everywhere. I have spent two years getting good at exactly that room.

What I learned

You cannot prevent failure at scale; you can only decide who finds out first. Sixteen months of events cured me of the idea that enough planning makes things not break. What's actually available to you is designing the degradation — what still works, what announces itself, and how fast the right person knows. Every system I've built since has a documented failure mode, and that habit came from a wifi outage, not a textbook.

Explaining a thing to someone who doesn't already agree is the only real test of understanding it. Fifty students taught me that I did not understand several things I could implement correctly. That's why I want a cohort, why I write my READMEs edges-first, and why the open-source work suits me — a maintainer who owes you nothing is just the hardest version of a student with a question.



Self-initiated · Solo · Shell · Markdown vault · Claude agent · macOS LaunchAgent · Hand-written HTML dashboard

Private repository, local demo. The only project here with no public URL — and it's in this portfolio anyway, because it's the one I built for a user I know everything about.

The Broken System

July 2026. A full-time internship with Slack pinging and tickets piling up, college deadlines, **three applications in flight**, a flight to book, and a personal life underneath all of it. Every open loop was being held in working memory, because there was nowhere else to put it.

The result wasn't laziness, and the data says so plainly: **120 LeetCode problems solved. GitHub days with 24 contributions.** The work was happening.

The result was **fog** — the mental static of too many open loops with no place to land. Every decision cost more than it should, because a background process was permanently running, checking whether something had been forgotten. Nothing was ever fully put down, so nothing was ever fully picked up.

The obvious diagnosis is a discipline problem and the obvious fix is a better habit. I'd tried that repeatedly. It doesn't work, for a reason worth naming:

Every capture tool I'd used charged its tax at exactly the wrong moment.

The thought arrives mid-task. Capturing it means opening an app, choosing a project, picking a tag, deciding whether it's a task or a note — and *deciding* is precisely the resource you had none of, which is why the thought was going to be lost.

So the tools built to reduce cognitive load were demanding cognitive load as their entry fee. Fog isn't cleared by thinking harder. It's cleared by writing things down, into a place that sorts, remembers, and shows you only what matters now.

Three Organs, Three Jobs

 THE VAULT	 THE AGENT	 THE PAGE
~/Life0S — plain markdown and JSON on my Mac	Claude, reading and writing the vault	dashboard.html , a render of the vault
One file per day, per project, per domain: journal/ , goals.md , projects/ , habits.md , fitness.md , applications.md , tickets.md , dates.md , notes.md	Infers intent from anything typed. Triage the capture queue. Pulls in the outside world each morning	Five surfaces: Today · Work · Life · Game · Manifest
Human-readable, greppable, portable, mine	Never interrogates. Hard-caps focus at three items	Opens, always, to exactly one thing
Law: archive, never delete · absolute dates only · the vault is the truth	Law: capture and processing are decoupled	Law: default-collapsed. Never show the whole house

The vault is the source of truth. Every page is only a render of it. If the agent and the dashboard both vanished tomorrow, the ten years of my life inside ~/Life0S would still open in any text editor.

How the System Works

The entire capture interface is one line, from whatever terminal I'm already in:

```
brain call maa about the flight
```

A shell function appends it to inbox/quick.md and **returns immediately**. No AI runs. Nothing loads. **Two seconds, zero judgment, sorted later.**

The intelligence happens afterwards, when the cost of thinking isn't being charged to me mid-task. The next agent run triages every queued line into the right file — tasks to the journal, ideas to ideas.md , dates to dates.md , protein to fitness.md — automatically.

And every morning at **8:23**, a macOS LaunchAgent fires a headless session that:

1. Drains and triages the capture queue
2. Pulls the world — weather, both GitHub accounts (open PRs, reviews, issues, contribution calendars via GraphQL), LeetCode counts and streak, calendar and reminders, Mac vitals, and the work-ticket ledger
3. Settles XP, audits badge conditions, rebuilds the learning graph
4. Rewrites the dashboard's data block and republishes it

Every source fails independently. A dead VPN never blocks the weather. The system degrades in pieces rather than collapsing.

I wake up and the day is already assembled.

The Design Decision — A game layer that cannot be cheated

There's an XP economy on top: check-in +10, focus item shipped +25, something learned +20, a LeetCode solve +30, a GitHub contribution +5, gym +40, protein target +20, the Sunday review +100. Levels unlock at $(n-1)^2 \times 100$. I'm **Fog Fighter, 570 XP**. Five of twelve badges unlocked — Genesis, Centurion, Medium Rare, Hard Mode, Green Wall.

The instinct is to let the user tick boxes. It's the wrong call, and it fails predictably.

Every gamified productivity app I've used eventually becomes a game *about the app*. You tick a box to move the number, the number stops corresponding to anything real, and within a fortnight you're maintaining a system instead of doing work. **A number that is cheap to move is a number you will quietly stop believing** — and you'll be right to.

So the rule is absolute: **XP is earned only from real, verified data. Never self-reported.** The LeetCode solve is read from LeetCode. The contribution is read from GitHub. The badge unlocks when the condition is true in the data, not when I say it is.

I cannot lie to it. That is the only reason I still believe it.

Which is the same principle as everything else in this portfolio, turned on myself. ATTEST refuses to post what it can't prove. Recall refuses to rank probabilistically. Kunal HQ won't award me points I didn't earn. I didn't plan that symmetry — I noticed it months later, looking at five projects side by side, and concluded that I only trust systems that are structurally incapable of flattering me.

The Interfaces

Five rooms, entered one at a time, never all at once.

Today (*always opens here*) — THE ONE THING banner, today's three focus items, a "waiting on you" digest, agenda and reminders, the player ring and daily quests. This *is* the day.

Work — tickets, including a **ghost work** column for the things that consume you without appearing anywhere. GitHub and Slack radar. The application pipeline, which currently has MATS and this Academy in it.

Life — habit heatmap and streaks, the learning graph, body metrics, the dreamlist, goals.

Game — trophy room, recent wins, the XP economy laid out honestly.

Manifest — the product poster. What this is, its principles, where it's going.

Inside each surface, cards **fold to one-line summaries** (**TICKETS** — 6 open · 3 ghost >). Today's cards open ready; every other room opens quiet, and the page remembers what I keep open.

Default-collapsed is law. The overwhelm this product exists to kill must never come from the product.

The look took **ten iterations** — bold, silver, beige, porcelain, sakura — before landing on serif-on-sand rose-gold with a drift of petals. A tool you open every morning of your life at 8am should not feel like a dashboard at work.

The Proof

It runs itself, every morning, on a schedule I don't touch, and it has for months. `hq` in any terminal opens it. The pinned tab reloads every ten minutes.

I use it every day. It runs my mornings. It's on v10.

That's the only metric I have and the only one that can't be gamed — I'm the entire user base, there's nobody to impress, abandonment would be instant and total, and I haven't abandoned it. `applications.md` is tracking this application while you read this page.

Why this matters for the Academy

Their first evaluation criterion is *"interest and motivation towards an intense 10-month program."* This is my honest answer to it.

July 2026 is the closest I've come to an intense sustained load — an internship, a degree, three applications, and a life, all at once — and I didn't handle it by working harder. I built the thing that made it holdable, and then I kept using it for nine months. **That's the evidence I'd offer that ten months in Bali wouldn't break me:** not that I'm tireless, but that when I'm overloaded my response is to build the system that clears the fog rather than to quietly drop things.

`applications.md` in the vault is tracking this application while you read this page.

What I Learned

Adoption is decided at the moment of capture and nowhere else. My first version had good organisation — structure, categories, a sensible schema — and I used it for about a week. The version that stuck costs two seconds and asks nothing, with every organising decision deferred to a moment when I'm not busy. That generalised immediately, and it's now the first question I ask of any interface: **find the moment the user has least to give, and make sure that's the moment you ask for nothing.**

Building for exactly one user taught me more about product than building for a hypothetical many. There's nowhere to hide. I couldn't excuse a feature by imagining someone who thinks differently, because there is no one else, and I couldn't declare success from usage numbers, because I'd know I was forcing myself. Every honest signal came from noticing what I quietly stopped doing. It's the most direct feedback loop I've had, and it left me deeply suspicious of any feature I can't name a specific person for.